Infraestrutura de software
Guia de Relatorios

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O objetivo desta atividade foi implementar um mecanismo de sincronização entre passageiros e carros de uma estação de metrô, utilizando threads POSIX (Pthreads). A implementação permite controlar a espera dos passageiros, a chegada dos carros e a autorização para embarque. O sistema garante que o carro aguarde o embarque dos passageiros autorizados antes de partir. A entrega contém os arquivos de implementação, cabeçalho, Makefile, programa de testes e documentação.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Implementar em linguagem C	OK	Verificar `sync_car.c` e `sync_car.h`.
Utilizar Pthreads para sincronização	OK	Verificar uso de mutexes e variáveis de condição no código.
Utilizar as funções de mutex e variáveis de condição exigidas	OK	Inspecionar as chamadas `pthread_mutex_*` e `pthread_cond_*`.
Não utilizar semáforos ou outras primitivas de sincronização.	OK	
Utilizar no máximo um mutex por estação.	OK	Verificar a estrutura `struct station` em `sync_car.h`.
Considerar que há no máximo um carro na estação por vez.	OK	
Permitir que vários passageiros sejam autorizados antes de qualquer um confirmar o embarque.	OK	Executar o `sync_car-runner.c`.

Evitar busy-wait.	OK	
Compilar utilizando Makefile e gerar o executável `sync_car`	OK	

3. Como Reproduzir

Sistema operacional: Ubuntu 24.04 LTS (WSL2).

Compilar:

```
make clean
make
```

Executar os testes:

```
./sync_car
```

Alternativamente, executar compilação e testes:

```
make test
```

Limpar os arquivos gerados:

```
make clean
```

A compilação utiliza o GCC, com suporte à biblioteca Pthreads, configurado no Makefile por meio da opção -pthread. Ao executar o programa de testes, a mensagem Looks good! indica que o runner concluiu a validação com sucesso.

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

4. Arquitetura (resumida)

Arquivos e responsabilidades

1. `sync_car.c` → Implementa as funções de sincronização da estação, controlando a chegada dos carros, a espera dos passageiros e a confirmação do embarque.
2. `sync_car.h` → Declara a estrutura `station`, suas variáveis compartilhadas e os protótipos das funções de sincronização.
3. `sync_car-runner.c` → Contém o programa de testes fornecido pelo professor, utilizado para verificar o comportamento da implementação.
4. `Makefile` → Automatiza a compilação do projeto, a execução dos testes e a limpeza dos arquivos gerados.
5. `README.md` → Documenta os arquivos do projeto e explica como compilar, executar e testar o programa.

Decisões técnicas:

1. Utilização de um único mutex por estação → O enunciado limita a implementação a um mutex por `struct station` → Todas as variáveis compartilhadas são protegidas pelo mesmo mutex, evitando condições de corrida durante a atualização dos contadores.
2. Utilização de duas variáveis de condição → Passageiros e carro precisam aguardar eventos diferentes → `car_available` acorda os passageiros quando há um carro disponível, enquanto `passenger_event` permite que o carro aguarde as confirmações de embarque.

3. Controle por geração de carro e quantidade de passageiros autorizados → Era necessário impedir que passageiros que chegassem durante um embarque fossem incluídos indevidamente no carro atual → `car_generation` identifica a chegada de um novo carro, enquanto `passengers_to_authorize` controla quantos passageiros ainda podem receber autorização para embarcar.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

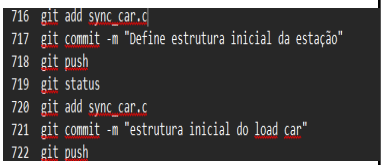
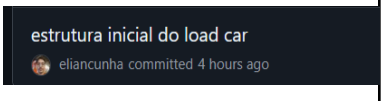
5.1 Estratégias da semana

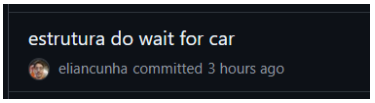
Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

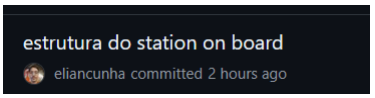
Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Mutex e variáveis de condição	Implementação da sincronização entre passageiros e carros utilizando um mutex e variáveis de condição Pthreads para controlar a espera, a autorização e a confirmação do embarque.	Não houve troca de estratégia.

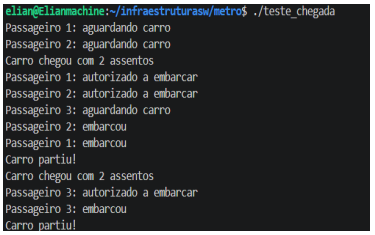
5.2 Diário de Tentativas

Obs: Esqueci de ativar o evidencias.log no desenvolvimento mas recuperei o histórico do terminal e usarei como evidência.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	s1	Criação da estrutura inicial da estação e implementação de `station_init` e load car.	Estrutura inicial criada e registrada no commit `Configura estrutura inicial do projeto.`		Dia: 4/11/26 16:30	Historico:  716 git add sync_car.c 717 git commit -m "Define estrutura inicial da estação" 718 git push 719 git status 720 git add sync_car.c 721 git commit -m "estrutura inicial do load car" 722 git push Commit:  estrutura inicial do load car eliancunha committed 4 hours ago

2	s1	Implementação de `station_wait_for_car`.	Etapa registrada no commit `estrutura do wait for car`		Dia 4/11/26 17:00(estimativa)	Historico:
						`git add sync_car.c`
						`git commit -m "estrutura do wait for car"`
						`git push`
						Commit:
						 

3	s1	Implementação de `station_on_board`	Etapa registrada no commit `estrutura do station on board`		Dia 4/11/26 18:00(estimativa)	Historico:
						`git add sync_car.c`
						`git commit -m "estrutura do station on board"`
						`git push`
						Commit:
						 

4	s1	Teste de chegada de passageiros durante o embarque, utilizando `teste_chegada.c`.	O teste foi compilado e executado. O resultado observado anteriormente demonstrou que passageiros que chegam durante o embarque aguardam o próximo carro		Dia 4/11/26 21:00	Resultados da execução registrados em `evidencias.log`. 
5	s1	Teste de capacidade com vários passageiros e carros, utilizando `teste_capacidade.c`.	O teste foi compilado e executado. O resultado observado foi a distribuição dos passageiros entre carros com capacidade limitada.		Dia 4/11/26 21:00	Resultados da execução registrados em `evidencias.log`.
6	s1	Teste de sinalização utilizando `teste_sinal.c`	O teste foi compilado e executado para verificar se um sinal sem a condição necessária não autorizava o passageiro a prosseguir.		Dia 4/11/26 21:00	Resultados da execução registrados em `evidencias.log`.
7	s1	Runner oficial	Compilação e execução final		Dia 4/11/26 21:00	resultado `Looks good!`, registrados em

			feita com sucesso			`evidencias.log`
8	s1	Repetição do runner	Registro das 20 execuções consecutivas com sucesso		Dia 4/11/26 21:00	registrados em `evidencias.log`.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

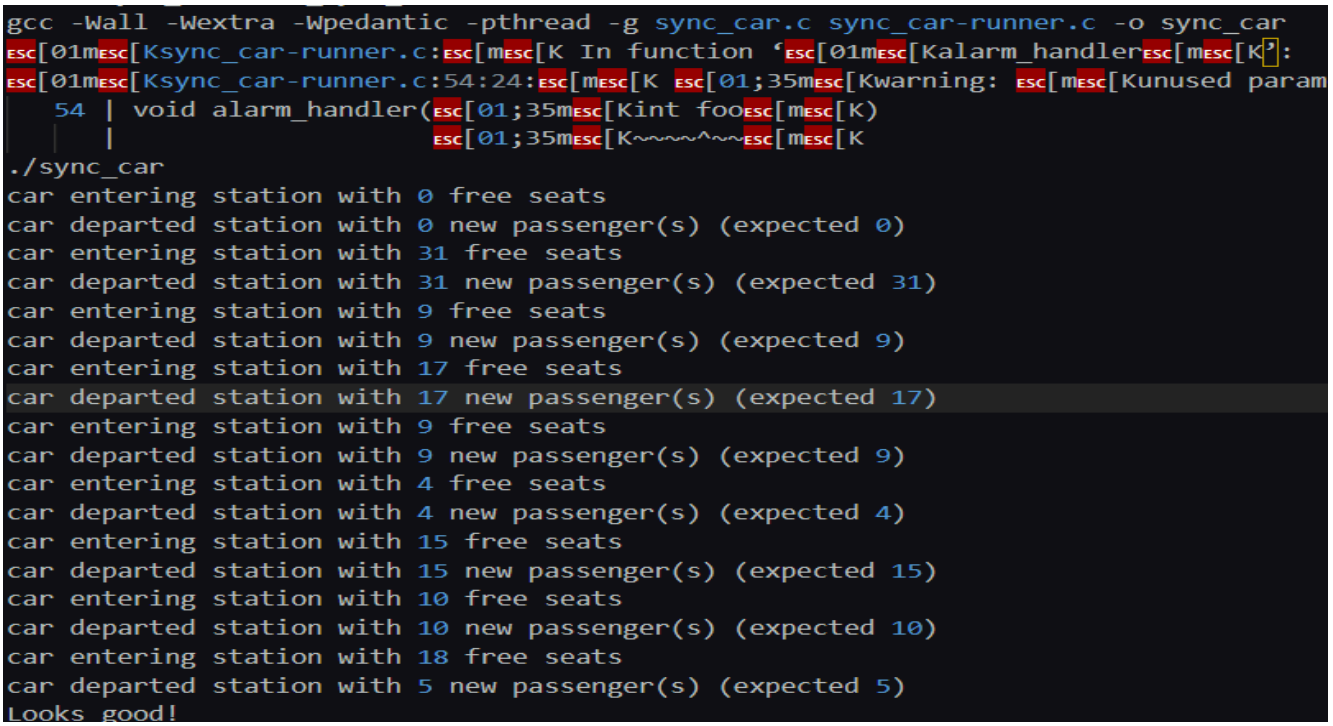
6.1 Log automático (faz o trabalho pesado por você)

Foi utilizado o comando script -a evidencias.log para registrar as atividades de validação realizadas no terminal. O arquivo contém os comandos executados, a compilação do projeto pelo Makefile e a saída do runner oficial, incluindo a mensagem Looks good além de testes pessoais adicionais feitos.

O registro foi incluído no pacote `.tar` da entrega para permitir a consulta das evidências de execução.

6.2 Prints

Runner oficial:



Repetição do runner:

```
for i in $(seq 1 20); do

    echo "Execução $i"

    ./sync_car || break

done
ESC[?2004l
===== 20 EXECUÇÕES DO RUNNER OFICIAL =====
Execução 1
car entering station with 33 free seats
car departed station with 33 new passenger(s) (expected 33)
car entering station with 37 free seats
car departed station with 37 new passenger(s) (expected 37)
car entering station with 18 free seats
car departed station with 18 new passenger(s) (expected 18)
car entering station with 4 free seats
car departed station with 4 new passenger(s) (expected 4)
car entering station with 14 free seats
car departed station with 8 new passenger(s) (expected 8)
Looks good!
Execução 2
car entering station with 28 free seats
car departed station with 28 new passenger(s) (expected 28)
car entering station with 28 free seats
car departed station with 28 new passenger(s) (expected 28)
car entering station with 21 free seats
car departed station with 21 new passenger(s) (expected 21)
car entering station with 10 free seats
car departed station with 10 new passenger(s) (expected 10)
car entering station with 21 free seats
```

...

```
Execução 19
car entering station with 38 free seats
car departed station with 38 new passenger(s) (expected 38)
car entering station with 37 free seats
car departed station with 37 new passenger(s) (expected 37)
car entering station with 20 free seats
car departed station with 20 new passenger(s) (expected 20)
car entering station with 4 free seats
car departed station with 4 new passenger(s) (expected 4)
car entering station with 26 free seats
car departed station with 1 new passenger(s) (expected 1)
Looks good!
Execução 20
car entering station with 24 free seats
car departed station with 24 new passenger(s) (expected 24)
car entering station with 19 free seats
car departed station with 19 new passenger(s) (expected 19)
car entering station with 3 free seats
car departed station with 3 new passenger(s) (expected 3)
car entering station with 16 free seats
car departed station with 16 new passenger(s) (expected 16)
car entering station with 0 free seats
car departed station with 0 new passenger(s) (expected 0)
car entering station with 9 free seats
car departed station with 9 new passenger(s) (expected 9)
car entering station with 9 free seats
car departed station with 9 new passenger(s) (expected 9)
car entering station with 16 free seats
car departed station with 16 new passenger(s) (expected 16)
car entering station with 37 free seats
car departed station with 4 new passenger(s) (expected 4)
Looks good!
```

7. Uso de IA

Onde usei IA (2–3 linhas): Durante o desenvolvimento do projeto, foi utilizado o ChatGPT (versão gratuita)

como ferramenta de apoio à aprendizagem e à implementação.

Prompts principais (liste, sem história):

inicial:"me ajude a organização desta implementação pedida, qual a organização do ambiente de trabalho necessária e suas ferramentas.Quais deveriam ser meus primeiros passos?"

Desenvolvimento: perguntas e respostas (conteudo educacional):

"A resposta é A. Explique por que esses contadores mudam na reserva, mas passengers_boarded não. " (exemplo de resposta correta)

"A resposta é A. Explique por que essa interpretação está incorreta. (exemplo de resposta incorreta)"

Sugestões:

"Estou na seção 5 adicionando os teste que fizemos, estou refazendo os testes para printar em evidencias mas alguns testes são muito longos para printar, você disse algo sobre refaze-los em um script, seria melhor no caso?Alguma sugestão?"

"fiz o esqueleto da função seguindo os passos dados e a minha duvida é sobre o incremento dos passageiros a bordo, precisamos de uma condição para adicionar? ou um loop como o while? Sinta-se livre para sugerir mais adições: void station_on_board(struct station *station){ pthread_mutex_lock(&station->lock); station->passengers_boarded ++; pthread_cond_signal(&station->car_available); pthread_mutex_unlock(&station->lock);} "

O que validei manualmente e como:

- Verifiquei a sincronização entre passageiros e carros por meio de testes personalizados, observando se passageiros que chegavam durante o embarque aguardavam o próximo carro
- Verifiquei o controle da capacidade dos carros utilizando um teste com cinco passageiros e veículos com dois assentos, observando a distribuição dos embarques.
- Verifiquei o comportamento das variáveis de condição por meio de um teste de sinal, observando se um passageiro permanecia aguardando enquanto não houvesse autorização para embarcar.
- Analisei a implementação para conferir o uso de mutexes, variáveis de condição e estruturas `while` nas esperas, evitando busy-wait e verificando novamente as condições de sincronização.
- Compilei o projeto utilizando o Makefile e executei o runner oficial, obtendo a mensagem `Looks good!`. Também repeti a execução 20 vezes para verificar a estabilidade da implementação.

8. Reflexão Final (5–8 linhas)

Se tivesse mais tempo, ampliaria a quantidade de testes, explorando mais combinações de passageiros, capacidades dos carros e ordens de chegada, além de cenários de concorrência mais variados.

Também iniciaria o registro com `script` desde o começo do desenvolvimento, mantendo evidências de cada etapa, tentativa e resultado. Isso facilitaria a elaboração do relatório e tornaria a documentação do processo mais completa.

Por fim, considero que a atividade contribuiu para desenvolver minha compreensão sobre sincronização de threads, programação concorrente e a importância de validar uma implementação por meio de testes.

Reflexão:

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	V
Rodei os testes e evidencias.log foi gerado	V

Tenho prints obrigatórios	V
Testei pelo menos um caso limite e um caso inválido	V
Preenchi a seção de uso de IA	V
Revisei o relatório e removi frases genéricas / vazias	V

10. Se eu tivesse mais 2 horas

Se tivesse mais duas horas para trabalhar no projeto, dedicaria parte desse tempo à criação e execução de testes adicionais, explorando diferentes quantidades de passageiros, capacidades dos carros e ordens de chegada.

Também procuraria testar cenários com maior concorrência entre threads, buscando identificar possíveis problemas de sincronização que não tenham sido contemplados pelos testes realizados.

Por fim, revisaria a implementação e a documentação para garantir que as decisões de sincronização estivessem bem explicadas e que os testes cobrissem os principais comportamentos exigidos pela atividade.